

Hanoi Kuleleri Uygulaması Raporu

❖ GİRİŞ

Bu projede, klasik bir algoritma problemi olan **Hanoi Kuleleri** sorununun çözümü C# dili kullanılarak gerçekleştirilmiştir. Kullanıcıdan alınan disk sayısına göre hangi disklerin hangi sırayla hangi çubuklara taşınması gerektiği ekrana yazdırılmaktadır. Bu sayede recursive (özyinelemeli) fonksiyonların çalışma mantığı uygulamalı olarak gösterilmiştir.

❖ PROBLEMİN TANIMI

Hanoi Kuleleri problemi 3 adet çubuk ve bu çubuklara dizilmiş farklı boyutlardaki disklerden oluşur. Amaç, tüm diskleri **A** çubuğundan **C** çubuğuna taşımaktır. Taşınırken aşağıdaki kurallara uyulmalıdır:

1. Aynı anda sadece bir disk taşınabilir.
2. Hiçbir zaman büyük bir disk küçük bir diskin üstüne konamaz.
3. Sadece üç çubuk (A, B, C) kullanılabilir.

❖ UYGULAMANIN AÇIKLAMASI

Hazırlanan C# uygulamasında, Main fonksiyonu kullanıcıdan disk sayısını alır ve ardından CozHanoi isimli recursive fonksiyonu çağırır. Bu fonksiyon sayesinde:

- Eğer sadece 1 disk varsa direkt hedefe taşınır.
- Birden fazla disk varsa, önce en üstteki diskler yardımcı çubuğa taşınır.
- Ardından en büyük disk hedefe yerleştirilir.
- Son olarak yardımcısındaki diskler hedefe taşınır.

Kodda A çubuğu kaynak, C çubuğu hedef, B çubuğu ise yardımcı olarak kullanılmıştır.

```
class HanoiKuleleri
```

```
{  
    static void Main()  
    {  
        Console.Write("Disk sayısını giriniz: ");  
        int diskSayisi = int.Parse(Console.ReadLine());  
        Console.WriteLine("\n--- Hanoi Kuleleri Çözümü ---\n");  
        CozHanoi(diskSayisi, 'A', 'C', 'B');  
    }  
  
    static void CozHanoi(int disk, char kaynak, char hedef, char yardimci)  
    {  
        if (disk == 1)  
        {  
            Console.WriteLine($"Disk 1'i {kaynak} çubuğundan {hedef} çubuğuna  
taşı");  
            return;  
        }  
  
        CozHanoi(disk - 1, kaynak, yardimci, hedef);  
  
        Console.WriteLine($"Disk {disk}'i {kaynak} çubuğundan {hedef}  
çubuğuna taşı");  
  
        CozHanoi(disk - 1, yardimci, hedef, kaynak);  
    }  
}
```

KODUN ÇIKTISI

Hamle No	Yapılan İşlem
1	Diski 1. iğneden (Sol) 3. iğneye (Sağ) taşı.
2	Diski 2. iğneden (Sol) 3. iğneye (Orta) taşı.
3	Diski 1. iğneden (Sağ) 3. iğneye (Orta) taşı.
4	Diski 3. iğneden (Sol) 3. iğneye (Sağ) taşı.
5	Diski 1. iğneden (Orta) 3. iğneye (Sol) taşı.
6	Diski 2. iğneden (Orta) 3. iğneye (Sağ) taşı.
7	Diski 1. iğneden (Sol) 3. iğneye (Sağ) taşı.
Toplam	En az hamle sayısı ($2^3 - 1$): 7